

Как да напишем `operators.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · friend, operator<<, сравнение, файлов изход

Какво ще направим

До упражнение 3 извеждахме така:

```
cout << " "; stud.Output(cout) << endl;
```

Работи, но не изглежда като C++. Вградените типове се извеждат просто с `cout << x` и няма причина нашите класове да са по-неудобни. След това упражнение ще пишем:

```
cout << stud << endl;
ofile << stud << endl;           // същият оператор, друг поток
cout << addr1 << " | " << addr2; // верижно
```

Изискване	Какво добавяме
1	<code>operator<<</code> за <code>CAddress</code>
2	частна функция <code>getYMD</code> — разлага ЕГН на година, месец и ден
3	<code>operator<<</code> за <code>CPerson</code>
4	приятелска функция <code>ConsoleInputEGN</code> — конзолен вход
5	<code>operator<<</code> за <code>CStudent</code>
6	оператори <code>==</code> , <code>!=</code> , <code><</code> , <code>>=</code> за сравнение по възраст
7–8	главна функция и извеждане във файл <code>Тема04.txt</code>

План на работата

- Нов проект и класовете от упражнение 3
- Защо `operator<<` не може да бъде член на класа
- `operator<<` за `CAddress` (изискване 1)
- Частната функция `getYMD` (изискване 2)
- `operator<<` за `CPerson` (изискване 3)
- `operator<<` за `CStudent` (изискване 5)
- Оператори за сравнение по възраст (изискване 6)
- `ConsoleInputEGN` (изискване 4)
- Главна функция и файлов изход (изисквания 7–8)

1 Нов проект и класовете от упражнение 3

Нов **Empty Project**, например `Тема4_Operators`, файл `operators.cpp`. Пренесете целите класове `CPerson`, `CAddress` и `CStudent` от упражнение 3 — с деструктора, копиращия конструктор, оператора `=` и динамичния масив от оценки.

ДОБАВЕТЕ КЪМ INCLUDE-ОВЕТЕ

```
#include <fstream>    // fstream – файлови потоци
#include <iomanip>      // setw, setfill
#include <sstream>      // ostringstream
```

Проверка: компилирайте, преди да сте променили каквото и да било.

2 Защо `operator<<` не може да бъде член

Първият въпрос е защо изобщо се пише *глобална* функция, а не член на класа както досега.

Когато напишете `a << b`, компилаторът търси две неща:

```
a.operator<<(b)          // член-функция на типа на ЛЯВАТА страна
operator<<(a, b)          // глобална функция с два параметъра
```

В нашия случай лявата страна е `cout` — обект от тип `ostream`. За да бъде `operator<<` член, той трябва да е член на `ostream`. Тоест трябва да влезем в стандартната библиотека и да добавим функция в `ostream`, което не е възможно.

Правилото. Когато лявата страна на оператора не е от вашия клас, операторът задължително е **глобална функция**. Затова `<<` и `>>` винаги се пишат глобално, а `=`, `[]`, `()` и `->` задължително са членове — езикът изрично го изисква.

АКО Е ГЛОБАЛНА, КАК ЩЕ СТИГНЕ ДО ЧАСТНИТЕ ЧЛЕНОВЕ?

Тук идва думата `friend`. Тя се пише **вътре в класа** и казва: „тази външна функция има право да вижда частните ми членове.“

```
class CAddress
{
private:
    string m_strCity;
    ...
public:
    friend ostream& operator<<(ostream& toStream, const CAddress& addr);
};
```

- 1 `friend` е **обявяване**, не дефиниция. Функцията не става член на класа — тя си остава глобална. Редът само ѝ дава разрешение.

- 2 Мястото в класа няма значение — `friend` обявяване в частната секция дава същите права. Приетата практика е да стои при публичните, защото функцията е част от публичния интерфейс.
- 3 **Приятелството не се наследява.** Приятел на `CPerson` не е автоматично приятел на `CStudent` и обратно. Това ще има значение в стъпка 6.
- 4 **Приятелството не е взаимно.** Ако `A` обяви `B` за приятел, `B` не получава достъп до `A` в обратната посока.

`friend` отваря дупка в капсулацията и затова се използва пестеливо. Тук е оправдано: без него операторът за извеждане би трябвало да минава през публичните `Get...` функции, а за някои класове такива просто няма смисъл да съществуват.

Всъщност за `CAddress` и `CPerson` публични `Get...` функции има и операторът би могъл да мине без `friend`. Пишем го с `friend`, защото условието го изисква изрично и защото за `CStudent` (стъпка 6) наистина няма друг начин да се стигне до `m_pdGrades`.

3 `operator<<` за `CAddress`

ОБЯВЕТЕ В КЛАСА, В ПУБЛИЧНАТА СЕКЦИЯ

```
friend ostream& operator<<(ostream& toStream, const CAddress& addr);
```

НАПИШЕТЕ СЛЕД КЛАСА — БЕЗ `CADDRESS::` ОТПРЕД

```
ostream& operator<<(ostream& toStream, const CAddress& addr)
{
    toStream << addr.m_strCity << " " << addr.m_strPostCode << ", " << addr.m_strStreet;
    return toStream;
}
```

Най-честата грешка: да се напише `ostream& CAddress::operator<<(...)`. Функцията **не е** член на класа — тя е глобална и `CAddress::` не ѝ се пише. Грешката е `C2039: 'operator <<': is not a member of 'CAddress'`.

ЧЕТИРИТЕ ЧАСТИ НА СИГНАТУРАТА

<code>ostream&</code> (връщане)	Връща се същият поток, за да работи верижното писане. <code>cout << a << b</code> значи <code>(cout << a) << b</code> — вътрешният израз трябва да даде обратно поток, върху който да се приложи вторият <code><<</code> . По референция , защото потоците не могат да се копират.
<code>ostream&</code> <code>toStream</code>	Не <code>const</code> — извеждането променя потока (движи позицията, вдига флагове за грешка).
<code>const</code> <code>CAddress&</code> <code>addr</code>	<code>const</code> , защото извеждането не променя обекта — и само така може да се изведе константен обект. Референция, за да не се копира целият обект при всяко извеждане.
без <code>endl</code>	Както при <code>Output</code> в упражнение 2: кой минава на нов ред решава извикващият.

ПРОБВАЙТЕ В MAIN

```
CAddress addrVarna("Studentska #1", "9010", "Varna");
CAddress addrSofia("Tsar Osvoboditel #12", "1000", "Sofia");
cout << " " << addrVarna << endl;
cout << "    верижно: " << addrVarna << " | " << addrSofia << endl;
```

Проверка: вторият ред извежда двата адреса на един ред. Ако се компилира, значи връщането на `ostream&` е написано правилно — с `void` нямаше да стане.

4 Частната функция `getYMD`

Условието задава: `void getYMD(const string& strEGN, int &iYear, int &iMonth, int &iDay) const;`

ОБЯВЕТЕ В ЧАСТНАТА СЕКЦИЯ НА CPERSON

```
private:
    void getYMD(const string& strEGN, int& iYear, int& iMonth, int& iDay) const;
```

НАПИШЕТЕ СЛЕД КЛАСА

```
// Месецът в ЕГН кодира и века: +20 => 18xx, +40 => 20xx.
// При невалиден месец връща iYear = 0 – това е сигналът за грешка.
void CPerson::getYMD(const string& strEGN, int& iYear, int& iMonth, int& iDay) const
{
    iYear = iMonth = iDay = 0;
    if (strEGN.length() != 10) return;

    int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
    int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
    int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

    if (nMonth >= 1 && nMonth <= 12) nYear += 1900;
    else if (nMonth >= 21 && nMonth <= 32) { nYear += 1800; nMonth -= 20; }
    else if (nMonth >= 41 && nMonth <= 52) { nYear += 2000; nMonth -= 40; }
    else return; // остава iYear = 0

    iYear = nYear;
    iMonth = nMonth;
    iDay = nDay;
}
```

ЗАЩО ТРИ РЕФЕРЕНТНИ ПАРАМЕТЪРА, А НЕ ВРЪЩАНА СТОЙНОСТ

Функцията трябва да даде **три** числа. Езикът позволява само една връщана стойност. Възможностите са:

изходни параметри <code>int&</code>	Това, което иска условието. Функцията пише направо в променливите на извикващия, защото <code>&</code> прави параметъра псевдоним, а не копие.
малка структура <code>struct Date</code>	По-чисто в модерен C++, но не е това, което иска условието.
три отделни функции	Три пъти същата работа за едно и също ЕГН.

Ако забравите `&` и напишете `int iYear`, функцията ще получи *копия*. Ще ги попълни прилежно и те ще изчезнат при връщането. Извикващият ще види непроменени стойности. Компиляторът не се оплаква — това е грешка, която се вижда само при изпълнение.

ДВЕТЕ ПУБЛИЧНИ ФУНКЦИИ НАД GETYMD

`getYMD` е частна — тя е вътрешна подробност. Отвън се предлагат две удобни функции:

```
// Дата на раждане като едно число ГГГГММДД: по-малко число = по-възрастен.
int CPerson::GetBirthKey() const
{
    int nYear, nMonth, nDay;
    getYMD(m_strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return 0;           // няма или невалидно ЕГН
    return nYear * 10000 + nMonth * 100 + nDay;
}

string CPerson::GetBirthDate() const
{
    int nYear, nMonth, nDay;
    getYMD(m_strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return "(няма)";

    // ostreamstream е поток, който пише в низ вместо на екрана.
    ostreamstream oss;
    oss << setfill('0') << setw(2) << nDay << "."
        << setw(2) << nMonth << "."
        << setw(4) << nYear;

    return oss.str();
}
```

Какво е ostreamstream? Поток, който вместо на екрана пише в низ в паметта. Използва се точно за такива случаи — форматиране на текст с всички средства на потоците (`setw`, `setfill`, `precision`), след което `.str()` дава готовия `string`.

Алтернативата `sprintf(szBuf, "%02d.%02d.%04d", ...)` също работи, но иска `#define _CRT_SECURE_NO_WARNINGS` при Visual Studio и буфер с ръчно избран размер. `ostreamstream` няма нито едното, нито другото.

Сега вие: пренапишете `isValid` така, че да използва `getYMD` вместо да разлага ЕГН сама. Проверката за месец вече е вътре в `getYMD` — остава да се провери дали `nYear != 0`, после броят дни в месеца и накрая контролната цифра. Сравнете с листинга.

5 operator<< за CPerson

ОБЯВЕТЕ В КЛАСА И НАПИШЕТЕ СЛЕД НЕГО

```
friend ostream& operator<<(ostream& toStream, const CPerson& pers);
```

```
ostream& operator<<(ostream& toStream, const CPerson& pers)
{
    toStream << pers.m_strName << " (" << pers.genderToString() << "), ЕГН: "
        << (pers.m_strEGN.empty() ? string("(няма)") : pers.m_strEGN)
        << ", роден: " << pers.GetBirthDate();
    return toStream;
}
```

Две функции с едно и също име `operator<<` ? Да — това е обикновено предефиниране (overloading).

Компиляторът избира по типа на втория параметър: `CAddress` или `CPerson`. Могат да съществуват колкото искате, стига типовете да са различни.

6 `operator<<` **3a** `CStudent`

Тук операторът трябва да изведе три неща: наследената личностна част, вградения адрес и собствените данни.

ОБЯВЕТЕ В КЛАСА И НАПИШЕТЕ СЛЕД НЕГО

```
friend ostream& operator<<(ostream& toStream, const CStudent& stud);
```

```
ostream& operator<<(ostream& toStream, const CStudent& stud)
{
    // Наследената част – чрез оператора на CPerson.
    toStream << static_cast<const CPerson&>(stud);

    toStream << ", ф.н.: " << stud.m_strFN
        << ", адрес: " << stud.m_address;    // чрез оператора на CAddress

    toStream << ", оценки: ";
    if (stud.m_nCount == 0)
        toStream << "(няма)";
    else
    {
        for (size_t i = 0; i < stud.m_nCount; i++)
            toStream << stud.m_pdGrades[i] << (i + 1 < stud.m_nCount ? " " : "");
        toStream << " | ср. успех: " << stud.GetAverage();
    }
    return toStream;
}
```

ЗАЩО `STATIC_CAST<CONST CPERSON&>(STUD)`

Това е най-важният ред в стъпката. Ако напишете просто `toStream << stud;`, функцията ще извика **себе си** — безкрайна рекурсия, същата грешка като при `Output` в упражнение 2.

Преобразуването казва: „гледай на този студент като на личност“. Тогава при избора на `operator<<` точно съвпада версията за `CPerson` и се извиква тя.

Защо е нужно изрично преобразуване? Студентът е личност, така че `const CPerson&` е допустим и без `static_cast`. Но при избор между две възможни функции компилаторът предпочита **точното** съвпадение — версията за `CStudent` — пред това, което иска преобразуване до базов клас. Тоест без `static_cast` се избира грешната. `static_cast` не заобикаля проверка, а само насочва избора.

Защо този оператор наистина има нужда от `friend`. Обърнете внимание на `stud.m_pdGrades[i]` и `stud.m_nCount` — тези членове са `private` в `CStudent` и за тях няма публична `Get` функция. Без `friend` редът е невъзможен.

Забележете и че операторът е приятел **само** на `CStudent`. До `m_strName` на `CPerson` той няма достъп — приятелството не се наследява. Затова наследената част се извежда през оператора на `CPerson`, а не направо.

7 Оператори за сравнение по възраст

Четири оператора: `==`, `!=`, `<`, `>=`. Сравняват се двама студенти по възраст.

ПЪРВО: КАК СЕ СРАВНЯВА ВЪЗРАСТ

За сравнението не е нужна днешната дата. Който е роден по-рано, е по-възрастен — винаги, независимо кой ден е днес. Затова сравняваме направо датите на раждане и резултатът не се променя с времето.

Това е и практически важно: ако сравнявахме чрез изчислена възраст спрямо „днес“, изходът на програмата щеше да е различен утре.

Използваме `GetBirthKey()` — числото ГГГГММДД. Внимавайте с посоката:

Ситуация	Дата	Ключ	Възраст
Георги	08.03.1975	19750308 — по-малък	по-голяма
Петър	22.07.2001	20010722 — по-голям	по-малка

По-малък ключ означава по-голяма възраст. Затова сравненията се обръщат.

ОБЯВЕТЕ В КЛАСА

```
friend bool operator==(const CStudent& studL, const CStudent& studR);
friend bool operator!=(const CStudent& studL, const CStudent& studR);
friend bool operator<(const CStudent& studL, const CStudent& studR);
friend bool operator>=(const CStudent& studL, const CStudent& studR);
```

НАПИШЕТЕ СЛЕД КЛАСА

```
bool operator==(const CStudent& studL, const CStudent& studR)
{
    return studL.GetBirthKey() == studR.GetBirthKey();
}

bool operator!=(const CStudent& studL, const CStudent& studR)
{

```

```

    return !(studL == studR);                // винаги през ==, за да не се разминат
}

bool operator<(const CStudent& studL, const CStudent& studR)
{
    return studL.GetBirthKey() > studR.GetBirthKey();
}

bool operator>=(const CStudent& studL, const CStudent& studR)
{
    return !(studL < studR);                // винаги през <
}

```

Разгледайте внимателно `operator<`. Знакът вътре е `>`, а не `<`. „Студентът отляво е по-малък по възраст“ значи „по-млад“ значи „роден по-късно“ значи „с по-голям ключ“. Не е печатна грешка — оставете коментара в кода, за да не я „поправи“ някой после.

ЗАЩО `!=` И `>=` СЕ ПИШАТ ПРЕЗ `==` И `<`

И двата биха могли да се напишат самостоятелно:

```
bool operator!=(...) { return studL.GetBirthKey() != studR.GetBirthKey(); } // също работи
```

Но тогава има **две независими** дефиниции на това кога двама студенти са равни. Ако утре промените `==` да сравнява и по име, `!=` ще остане по стара логика и двете ще си противоречат: и `a == b`, и `a != b` ще дават истина едновременно.

Като се напише `!(studL == studR)`, това е невъзможно по построение. Правилото: **изразявайте всеки оператор за сравнение чрез един основен**. Тук основните са `==` и `<`; останалите се извеждат от тях.

Защо тези оператори са глобални, а не членове? Тук лявата страна е от нашия клас, така че членове биха били възможни: `bool CStudent::operator<(const CStudent&) const`. Условието обаче ги иска глобални и приятелски — и това има едно реално предимство: при глобална версия двата операнда се третираат еднакво, включително при неявни преобразувания. При член-функция лявата страна не се преобразува. В упражнение 5 ще напишете нарочно **член-оператор** `>`, за да се види разликата.

8 ConsoleInputEGN

Условието задава: `friend void ConsoleInputEGN(CStudent& stud)`.

ОБЯВЕТЕ В КЛАСА И НАПИШЕТЕ СЛЕД НЕГО

```
friend void ConsoleInputEGN(CStudent& stud);
```

```

void ConsoleInputEGN(CStudent& stud)
{
    cout << " Въведете ЕГН за " << stud.m_strName << ": ";

    string strEGN;
    if (!(cin >> strEGN))
    {

```

```

        cout << endl << " (няма въведени данни – ЕГН остава " << stud.m_strEGN << ")" << endl;
        return;
    }

    if (stud.isValid(strEGN))
    {
        stud.m_strEGN = strEGN; // достъп до protected член – позволено
        cout << " Прието. Нова дата на раждане: " << stud.GetBirthDate() << endl;
    }
    else
    {
        cout << " Невалидно ЕГН. Остава старото: " << stud.m_strEGN << endl;
    }
}

```

`CStudent& stud`

Без `const` — за разлика от всички дотук. Функцията е длъжна да промени обекта, затова параметърът не може да е константен. Без `&` щеше да промени копие и нищо нямаше да се случи.

`if (!(cin >> strEGN))`

Четенето от поток връща самия поток, а потокът в логически контекст дава „истина“, ако четенето е успяло. Проверката улавя край на входа (`Ctrl` + `Z` под Windows) и предпазва от безкраен цикъл.

`stud.m_strEGN = strEGN;`

Директен запис в защитения член — възможен благодарение на `friend`. Проверката вече е направена ред по-горе.

А защо изобщо е нужен `friend`, щом има `SetEGN`? В този конкретен случай не е строго нужен — `stud.SetEGN(strEGN)` върши същото. Условието обаче изисква приятелска функция, а и примерът показва как се пише такава, когато класът няма подходящ мутатор.

9 Главна функция и файлов изход

9.1 Тест на трите оператора за извеждане

```

cout.setf(ios::fixed);
cout.precision(2);

cout << " " << addrVarna << endl; // CAddress
CPerson pers1("Иван Петров", "мъж", "7503081239");
cout << " " << pers1 << endl; // CPerson
CStudent stud0ld("Георги Стоянов", addrVarna, "7503081239", "21621301");
cout << " " << stud0ld << endl; // CStudent

```

9.2 Данни за теста на сравненията

ЕГН-тата от условието (`1111111110` и `1111111104`) не вършат работа тук — и двете кодират една и съща дата, 11.11.1911. За да се види разликата между операторите, трябва валидни ЕГН с **различни** дати:

ЕГН	Дата на раждане	Роля в теста
7503081239	08.03.1975	най-възрастен

8512034566	03.12.1985	среден
9005157895	15.05.1990	две различни ЕГН, една дата — за теста на ==
9005157909	15.05.1990	
0147222342	22.07.2001	най-млад

И петте минават валидизатора от упражнение 1. Проверете го — това е добър тест и за него.

9.3 Тест на сравненията

```
cout << " studSame == studTwin (една дата): " << (studSame == studTwin ? "да" : "не") <<
endl;
cout << " studOld != studYng: " << (studOld != studYng ? "да" : "не") <<
endl;
cout << " studYng < studOld (по-млад): " << (studYng < studOld ? "да" : "не") <<
endl;
cout << " studOld >= studYng: " << (studOld >= studYng ? "да" : "не") <<
endl;
```

9.4 Подредба по възраст — операторът върши работа

```
CStudent arrStud[5] = { studYng, studSame, studOld, studTwin, studMid };
const int nCount = 5;
for (int i = 0; i < nCount - 1; i++)
    for (int j = 0; j < nCount - 1 - i; j++)
        if (arrStud[j] < arrStud[j + 1]) // ляв е по-млад -> разменяме
        {
            CStudent tmp = arrStud[j];
            arrStud[j] = arrStud[j + 1];
            arrStud[j + 1] = tmp;
        }
```

Тези осем реда използват всичко дотук наведнъж. `arrStud[j] < arrStud[j+1]` — операторът от стъпка 7. `CStudent tmp = arrStud[j];` — копиращият конструктор от упражнение 3. Двете присвоявания — операторът `=` оттам. Ако правилото на тримата беше сгрешено, точно тази размяна щеше да повреди паметта.

9.5 Извеждане във файл

```
fstream oFile("Тема04.txt", ios_base::out);
if (!oFile)
{
    cout << " Файлът не може да бъде отворен за запис!" << endl;
}
else
{
    oFile.setf(ios::fixed);
    oFile.precision(2);

    oFile << "Студенти, подредени по възраст" << endl;
    oFile << "======" << endl;
    for (int i = 0; i < nCount; i++)
```

```
oFile << (i + 1) << ". " << arrStud[i] << endl;    // СЪЩИЯТ оператор <<

oFile.close();
}
```

Ето заради какво беше цялото упражнение. Не е написан нито ред нов код за записване във файл.

Операторът приема `ostream&`, а `fstream` е наследник на `ostream` — затова същата функция работи и за файл.

Сравнете с подхода „Print() с `cout` вътре“ от тема 1: там за файл трябваше втора функция, а за трети вид изход — трета.

Забележете и че форматирането (`setf`, `precision`) се задава отделно за всеки поток. Настройката на `cout` не важи за `oFile`.

Проверката `if (!oFile)` не се пропуска. Отварянето за запис се проваля при липса на права, при заключен от друга програма файл или при несъществуваща папка. Без проверката програмата ще „запише“ успешно в нищото и няма да каже нищо.

Къде се появява файлът? В работната папка на проекта — не в `Debug`, а в папката с `.vcxproj`. Ако не го намирате, потърсете по име в Solution Explorer → Show All Files.

9.6 Проверка на записаното

```
fstream iFile("Tema04.txt", ios_base::in);
if (iFile)
{
    string strLine;
    while (getline(iFile, strLine))
        cout << " " << strLine << endl;
    iFile.close();
}
```

`getline` чете цял ред заедно с интервалите и връща „лъжа“ в края на файла — затова е естественото условие на цикъла.

9.7 Конзолният вход

```
cout << " преди: " << studYng << endl;
ConsoleInputEGN(studYng);
cout << " след: " << studYng << endl;
```

Оставя се последна, защото спира програмата в очакване на вход. Въведете валидно ЕГН (например `9005157895`) и вижте как се променя редът „роден:“. После пробвайте с невалидно — старото трябва да се запази.

Компилиране и стартиране

Клавиши	Действие
---------	----------

Ctrl + Shift + B

Build — само компилира.

F5

Start Debugging.

Ctrl + F5

Start Without Debugging — задържа прозореца сам.

```
cl /EHsc /std:c++17 /utf-8 operators.cpp  
operators.exe
```

Кирилица под Windows

Ако вместо български букви виждате `???` : файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001* (File → Save As → стрелката до Save → Save with Encoding); компилаторът получава флаг `/utf-8` ; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main` (нужен е `#include <cstdlib>`).

Файлът `Тема04.txt` също се записва в UTF-8. Notepad го отваря правилно. По-старите редактори може да покажат кирилицата като безсмислени символи — това не значи, че програмата е сгрешила.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 4: Приятелски функции и оператори.
// Надгражда класовете CPerson, CAddress и CStudent от упражнение 3:
// функциите Output се заменят с глобални оператори <<, добавят се
// оператори за сравнение по възраст и извеждане във файл.

#include <cctype>      // isdigit
#include <cstdio>      // getchar
#include <fstream>     // fstream
#include <iomanip>      // setw, setfill
#include <iostream>
#include <sstream>      // ostringstream
#include <string>
using namespace std;

enum Gender
{
    MALE,           // мъж
    FEMALE,         // жена
    UNKNOWN         // неопределен
};

// =====
// CPerson
// =====
class CPerson
{
protected:
    string m_strName;
    Gender m_gender;
    string m_strEGN;

// 2. Частна функция: разлага ЕГН на година, месец и ден.
// Частна е, защото е вътрешна подробност – отвън се ползва GetBirthKey.
private:
    void getYMD(const string& strEGN, int& iYear, int& iMonth, int& iDay) const;

public:
    CPerson();
    CPerson(const string& strName, const string& strGender, const string& strEGN);
    CPerson(const CPerson& pers);

    void SetName(const string& strName);
    void SetGender(Gender gender);
    void SetGender(const string& strGender);
    bool SetEGN(const string& strEGN);

    const string& GetName() const;
    Gender GetGender() const;
    const string& GetEGN() const;

    Gender stringToGender(const string& strGender) const;
    string genderToString() const;
```

```

bool isValid(const string& strEGN);

// Дата на раждане като едно число ГГГГММДД: по-малко число = по-възрастен.
int GetBirthKey() const;
string GetBirthDate() const;           // "08.03.1975"

// 3. Глобална приятелска функция (оператор) за извеждане
friend ostream& operator<<(ostream& toStream, const CPerson& pers);
};

CPerson::CPerson()
    : m_strName("Неизвестен"), m_gender(UNKNOWN), m_strEGN("") {}

CPerson::CPerson(const string& strName, const string& strGender, const string& strEGN)
    : m_strName(strName), m_gender(UNKNOWN), m_strEGN("")
{
    m_gender = stringToGender(strGender);
    SetEGN(strEGN);
}

CPerson::CPerson(const CPerson& pers)
    : m_strName(pers.m_strName), m_gender(pers.m_gender), m_strEGN(pers.m_strEGN) {}

void CPerson::SetName(const string& strName)    { m_strName = strName; }
void CPerson::SetGender(Gender gender)         { m_gender = gender; }
void CPerson::SetGender(const string& strGender) { m_gender = stringToGender(strGender); }

bool CPerson::SetEGN(const string& strEGN)
{
    if (!isValid(strEGN)) return false;
    m_strEGN = strEGN;
    return true;
}

const string& CPerson::GetName() const { return m_strName; }
Gender CPerson::GetGender() const { return m_gender; }
const string& CPerson::GetEGN() const { return m_strEGN; }

Gender CPerson::stringToGender(const string& strGender) const
{
    if (strGender == "мъж" || strGender == "М" || strGender == "male") return MALE;
    if (strGender == "жена" || strGender == "Ж" || strGender == "female") return FEMALE;
    return UNKNOWN;
}

string CPerson::genderToString() const
{
    switch (m_gender)
    {
        case MALE: return "мъж";
        case FEMALE: return "жена";
        default: return "неопределен";
    }
}

bool CPerson::isValid(const string& strEGN)
{
    if (strEGN.length() != 10) return false;
    for (size_t i = 0; i < strEGN.length(); i++)

```

```

        if (!isdigit(static_cast<unsigned char>(strEGN[i]))) return false;

    int nYear, nMonth, nDay;
    getYMD(strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return false;           // getYMD сигнализира грешен месец с 0

    int arrDays[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
    bool bLeap = (nYear % 4 == 0 && nYear % 100 != 0) || (nYear % 400 == 0);
    if (nMonth == 2 && bLeap) arrDays[1] = 29;
    if (nDay < 1 || nDay > arrDays[nMonth - 1]) return false;

    const int arrWeights[9] = { 2, 4, 8, 5, 10, 9, 7, 3, 6 };
    int nSum = 0;
    for (int i = 0; i < 9; i++) nSum += (strEGN[i] - '0') * arrWeights[i];

    int nCheck = nSum % 11;
    if (nCheck == 10) nCheck = 0;
    return nCheck == (strEGN[9] - '0');
}

// ----- 2. Частната функция getYMD -----
// Месецът в ЕГН кодира и века: +20 => 18xx, +40 => 20xx.
// При невалиден месец връща iYear = 0 - това е сигналът за грешка.
void CPerson::getYMD(const string& strEGN, int& iYear, int& iMonth, int& iDay) const
{
    iYear = iMonth = iDay = 0;
    if (strEGN.length() != 10) return;

    int nYear = (strEGN[0] - '0') * 10 + (strEGN[1] - '0');
    int nMonth = (strEGN[2] - '0') * 10 + (strEGN[3] - '0');
    int nDay = (strEGN[4] - '0') * 10 + (strEGN[5] - '0');

    if (nMonth >= 1 && nMonth <= 12) nYear += 1900;
    else if (nMonth >= 21 && nMonth <= 32) { nYear += 1800; nMonth -= 20; }
    else if (nMonth >= 41 && nMonth <= 52) { nYear += 2000; nMonth -= 40; }
    else return;           // остава iYear = 0

    iYear = nYear;
    iMonth = nMonth;
    iDay = nDay;
}

int CPerson::GetBirthKey() const
{
    int nYear, nMonth, nDay;
    getYMD(m_strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return 0;           // няма или невалидно ЕГН
    return nYear * 10000 + nMonth * 100 + nDay;
}

string CPerson::GetBirthDate() const
{
    int nYear, nMonth, nDay;
    getYMD(m_strEGN, nYear, nMonth, nDay);
    if (nYear == 0) return "(няма)";

    // ostringstream е поток, който пише в низ вместо на екрана.
    ostringstream oss;
    oss << setfill('0') << setw(2) << nDay << "."

```

```

        << setw(2) << nMonth << "."
        << setw(4) << nYear;

    return oss.str();
}

// ----- 3. Оператор за извеждане на личност -----
ostream& operator<<(ostream& toStream, const CPerson& pers)
{
    toStream << pers.m_strName << " (" << pers.genderToString() << "), ЕГН: "
        << (pers.m_strEGN.empty() ? string("(няма)") : pers.m_strEGN)
        << ", роден: " << pers.GetBirthDate();
    return toStream;
}

// =====
//  CAddress
// =====
class CAddress
{
private:
    string m_strStreet;
    string m_strPostCode;
    string m_strCity;

public:
    CAddress();
    CAddress(const string& strStreet, const string& strPostCode, const string& strCity);
    CAddress(const CAddress& addr);

    const string& GetStreet() const;
    const string& GetPostCode() const;
    const string& GetCity() const;
    void SetStreet(const string& strStreet);
    void SetPostCode(const string& strPostCode);
    void SetCity(const string& strCity);

    // 1. Глобална приятелска функция (оператор) за извеждане
    friend ostream& operator<<(ostream& toStream, const CAddress& addr);
};

CAddress::CAddress()
    : m_strStreet("(няма улица)", m_strPostCode("0000"), m_strCity("(няма град)") {}

CAddress::CAddress(const string& strStreet, const string& strPostCode, const string& strCity)
    : m_strStreet(strStreet), m_strPostCode(strPostCode), m_strCity(strCity) {}

CAddress::CAddress(const CAddress& addr)
    : m_strStreet(addr.m_strStreet), m_strPostCode(addr.m_strPostCode),
      m_strCity(addr.m_strCity) {}

const string& CAddress::GetStreet() const    { return m_strStreet; }
const string& CAddress::GetPostCode() const { return m_strPostCode; }
const string& CAddress::GetCity() const     { return m_strCity; }

void CAddress::SetStreet(const string& strStreet)    { m_strStreet = strStreet; }
void CAddress::SetPostCode(const string& strPostCode) { m_strPostCode = strPostCode; }
void CAddress::SetCity(const string& strCity)        { m_strCity = strCity; }

// ----- 1. Оператор за извеждане на адрес -----

```

```

ostream& operator<<(ostream& toStream, const CAddress& addr)
{
    toStream << addr.m_strCity << " " << addr.m_strPostCode << ", " << addr.m_strStreet;
    return toStream;
}

// =====
// CStudent
// =====
class CStudent : public CPerson
{
private:
    string    m_strFN;
    CAddress  m_address;

    double*   m_pdGrades;
    size_t    m_nCount;
    size_t    m_nCapacity;

    void grow();

public:
    CStudent();
    CStudent(const string& strName, const CAddress& addr, const string& strEGN);
    CStudent(const string& strName, const CAddress& addr, const string& strEGN,
              const string& strFN);
    CStudent(const CStudent& stud);
    CStudent& operator=(const CStudent& stud);
    ~CStudent();

    void    AddGrade(double dGrade);
    double  GetGrade(size_t nIndex) const;
    size_t  GetGradeCount() const;
    double  GetAverage() const;

    const string&  GetFN() const;
    const CAddress& GetAddress() const;
    void SetFN(const string& strFN);
    void SetAddress(const CAddress& addr);

    // 4. Приятелска функция за конзолен вход към ЕГН
    friend void ConsoleInputEGN(CStudent& stud);

    // 5. Приятелски оператор за извеждане
    friend ostream& operator<<(ostream& toStream, const CStudent& stud);

    // 6. Приятелски оператори за сравнение ПО ВЪЗРАСТ
    friend bool operator==(const CStudent& studL, const CStudent& studR);
    friend bool operator!=(const CStudent& studL, const CStudent& studR);
    friend bool operator< (const CStudent& studL, const CStudent& studR);
    friend bool operator>=(const CStudent& studL, const CStudent& studR);
};

void CStudent::grow()
{
    size_t nNew = (m_nCapacity == 0) ? 4 : m_nCapacity * 2;
    double* pdNew = new double[nNew];
    for (size_t i = 0; i < m_nCount; i++) pdNew[i] = m_pdGrades[i];
    delete[] m_pdGrades;
}

```

```

        m_pdGrades = pdNew;
        m_nCapacity = nNew;
    }

CStudent::CStudent()
    : CPerson(), m_strFN("(няма)", m_address(),
        m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN)
    : CPerson(strName, "неопределен", strEGN), m_strFN("(няма)", m_address(addr),
        m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

CStudent::CStudent(const string& strName, const CAddress& addr, const string& strEGN,
                    const string& strFN)
    : CPerson(strName, "неопределен", strEGN), m_strFN(strFN), m_address(addr),
        m_pdGrades(nullptr), m_nCount(0), m_nCapacity(0) {}

CStudent::CStudent(const CStudent& stud)
    : CPerson(stud), m_strFN(stud.m_strFN), m_address(stud.m_address),
        m_pdGrades(nullptr), m_nCount(stud.m_nCount), m_nCapacity(stud.m_nCapacity)
{
    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity];
        for (size_t i = 0; i < m_nCount; i++) m_pdGrades[i] = stud.m_pdGrades[i];
    }
}

CStudent& CStudent::operator=(const CStudent& stud)
{
    if (this == &stud) return *this;

    CPerson::operator=(stud);
    m_strFN = stud.m_strFN;
    m_address = stud.m_address;

    delete[] m_pdGrades;
    m_pdGrades = nullptr;

    m_nCount = stud.m_nCount;
    m_nCapacity = stud.m_nCapacity;
    if (m_nCapacity > 0)
    {
        m_pdGrades = new double[m_nCapacity];
        for (size_t i = 0; i < m_nCount; i++) m_pdGrades[i] = stud.m_pdGrades[i];
    }
    return *this;
}

CStudent::~~CStudent()
{
    delete[] m_pdGrades;
    m_pdGrades = nullptr;
}

void CStudent::AddGrade(double dGrade)
{
    if (dGrade < 2.0 || dGrade > 6.0) return;
    if (m_nCount == m_nCapacity) grow();
}

```

```

        m_pdGrades[m_nCount] = dGrade;
        m_nCount++;
    }

double CStudent::GetGrade(size_t nIndex) const
{
    if (nIndex >= m_nCount) return 0.0;
    return m_pdGrades[nIndex];
}

size_t CStudent::GetGradeCount() const { return m_nCount; }

double CStudent::GetAverage() const
{
    if (m_nCount == 0) return 0.0;
    double dSum = 0.0;
    for (size_t i = 0; i < m_nCount; i++) dSum += m_pdGrades[i];
    return dSum / m_nCount;
}

const string& CStudent::GetFN() const { return m_strFN; }
const CAddress& CStudent::GetAddress() const { return m_address; }

void CStudent::SetFN(const string& strFN) { m_strFN = strFN; }
void CStudent::SetAddress(const CAddress& addr) { m_address = addr; }

// ----- 4. Приятелска функция за конзолен вход -----
// Приятелска е, за да пише направо в m_strEGN след собствената си проверка.
void ConsoleInputEGN(CStudent& stud)
{
    cout << " Въведете ЕГН за " << stud.m_strName << ": ";

    string strEGN;
    if (!(cin >> strEGN))
    {
        cout << endl << " (няма въведени данни – ЕГН остава " << stud.m_strEGN << ")" << endl;
        return;
    }

    if (stud.isValid(strEGN))
    {
        stud.m_strEGN = strEGN; // достъп до protected член – позволено
        cout << " Прието. Нова дата на раждане: " << stud.GetBirthDate() << endl;
    }
    else
    {
        cout << " Невалидно ЕГН. Остава старото: " << stud.m_strEGN << endl;
    }
}

// ----- 5. Оператор за извеждане на студент -----
ostream& operator<<(ostream& toStream, const CStudent& stud)
{
    // Наследената част – чрез оператора на CPerson.
    toStream << static_cast<const CPerson&>(stud);

    toStream << ", ф.н.: " << stud.m_strFN
        << ", адрес: " << stud.m_address; // чрез оператора на CAddress
}

```

```

toStream << ", оценки: ";
if (stud.m_nCount == 0)
    toStream << "(няма)";
else
{
    for (size_t i = 0; i < stud.m_nCount; i++)
        toStream << stud.m_pdGrades[i] << (i + 1 < stud.m_nCount ? " " : "");
    toStream << " | ср. успех: " << stud.GetAverage();
}
return toStream;
}

// ----- 6. Оператори за сравнение по възраст -----
// GetBirthKey() е ГГГГММДД. По-МАЛКО число = по-РАНО роден = по-ВЪЗРАСТЕН.
// Затова "по-малък по възраст" (по-млад) означава по-ГОЛЯМ ключ.
bool operator==(const CStudent& studL, const CStudent& studR)
{
    return studL.GetBirthKey() == studR.GetBirthKey();
}

bool operator!=(const CStudent& studL, const CStudent& studR)
{
    return !(studL == studR);           // винаги през ==, за да не се разминат
}

bool operator<(const CStudent& studL, const CStudent& studR)
{
    return studL.GetBirthKey() > studR.GetBirthKey();
}

bool operator>=(const CStudent& studL, const CStudent& studR)
{
    return !(studL < studR);           // винаги през <
}

// =====
// 7-8. Главна функция
// =====
int main()
{
    cout.setf(ios::fixed);
    cout.precision(2);

    CAddress addrVarna("Studentska #1", "9010", "Varna");
    CAddress addrSofia("Tsar Osvoboditel #12", "1000", "Sofia");

    // ===== 1. Оператор << за CAddress =====
    cout << "===== 1. Оператор << за CAddress =====" << endl;
    CAddress addrDefault;
    cout << "    addrVarna:    " << addrVarna << endl;
    cout << "    addrSofia:    " << addrSofia << endl;
    cout << "    addrDefault: " << addrDefault << endl;

    // Верижно извеждане – двата адреса на един ред
    cout << "    верижно: " << addrVarna << " | " << addrSofia << endl;

    // ===== 2-3. getYMD и оператор << за CPerson =====
    cout << endl << "===== 2-3. Оператор << за CPerson =====" << endl;
    CPerson pers1("Иван Петров", "Мъж", "7503081239");

```

```

CPerson pers2("Мария Иванова", "жена", "0147222342");
CPerson pers3;
cout << " " << pers1 << endl;
cout << " " << pers2 << endl;
cout << " " << pers3 << endl;

cout << "---- Разлагане на ЕГН (през GetBirthDate) ----" << endl;
const char* arrEGN[] = { "7503081239", "8512034566", "9005157895", "0147222342" };
CPerson persTest;
for (size_t i = 0; i < sizeof(arrEGN) / sizeof(arrEGN[0]); i++)
{
    persTest.SetEGN(arrEGN[i]);
    cout << " " << arrEGN[i] << " -> " << persTest.GetBirthDate()
        << " (ключ " << persTest.GetBirthKey() << ")" << endl;
}

// ===== 5. Оператор << за CStudent =====
cout << endl << "===== 5. Оператор << за CStudent =====" << endl;
CStudent studOld("Георги Стоянов", addrVarna, "7503081239", "21621301");
CStudent studMid("Елена Димитрова", addrSofia, "8512034566", "21621302");
CStudent studYng("Петър Колев", addrVarna, "0147222342", "21621303");
CStudent studSame("Анна Тодорова", addrSofia, "9005157895", "21621304");
CStudent studTwin("Стефан Николов", addrVarna, "9005157909", "21621305");

studOld.SetGender("мъж"); studOld.AddGrade(5.50); studOld.AddGrade(4.00);
studMid.SetGender("жена"); studMid.AddGrade(6.00); studMid.AddGrade(5.00);
studYng.SetGender("мъж"); studYng.AddGrade(3.00);
studSame.SetGender("жена"); studSame.AddGrade(4.50);
studTwin.SetGender("мъж");

cout << " " << studOld << endl;
cout << " " << studMid << endl;
cout << " " << studYng << endl;

// ===== 6. Оператори за сравнение по възраст =====
cout << endl << "===== 6. Сравнение по възраст =====" << endl;
cout << " studOld роден " << studOld.GetBirthDate() << endl;
cout << " studMid роден " << studMid.GetBirthDate() << endl;
cout << " studYng роден " << studYng.GetBirthDate() << endl;
cout << " studSame роден " << studSame.GetBirthDate() << endl;
cout << " studTwin роден " << studTwin.GetBirthDate() << endl;

cout << "---- Резултати ----" << endl;
cout << " studSame == studTwin (една дата): " << (studSame == studTwin ? "да" : "не") <<
endl;
cout << " studOld == studYng: " << (studOld == studYng ? "да" : "не") <<
endl;
cout << " studOld != studYng: " << (studOld != studYng ? "да" : "не") <<
endl;
cout << " studYng < studOld (по-млад): " << (studYng < studOld ? "да" : "не") <<
endl;
cout << " studOld < studYng: " << (studOld < studYng ? "да" : "не") <<
endl;
cout << " studOld >= studYng (по-възрастен или равен): "
    << (studOld >= studYng ? "да" : "не") << endl;
cout << " studSame >= studTwin (равни): " << (studSame >= studTwin ? "да" : "не") <<
endl;

// Сортиране по възраст - от най-възрастния към най-младия

```

```

cout << "---- Подредба по възраст (най-възрастният първи) ----" << endl;
CStudent arrStud[5] = { studYng, studSame, studOld, studTwin, studMid };
const int nCount = 5;
for (int i = 0; i < nCount - 1; i++)
    for (int j = 0; j < nCount - 1 - i; j++)
        if (arrStud[j] < arrStud[j + 1]) // ляв е по-млад -> разменяме
        {
            CStudent tmp = arrStud[j];
            arrStud[j] = arrStud[j + 1];
            arrStud[j + 1] = tmp;
        }
for (int i = 0; i < nCount; i++)
    cout << " " << (i + 1) << ". " << arrStud[i].GetName()
        << " - " << arrStud[i].GetBirthDate() << endl;

// ===== 8. Извеждане във файлов поток =====
cout << endl << "===== 8. Извеждане във файл Tema04.txt =====" << endl;
fstream oFile("Tema04.txt", ios_base::out);
if (!oFile)
{
    cout << " Файлът не може да бъде отворен за запис!" << endl;
}
else
{
    oFile.setf(ios::fixed);
    oFile.precision(2);

    oFile << "Студенти, подредени по възраст" << endl;
    oFile << "===== " << endl;
    for (int i = 0; i < nCount; i++)
        oFile << (i + 1) << ". " << arrStud[i] << endl; // СъЩИЯТ оператор <<

    oFile.close();
    cout << " Записани " << nCount << " реда. Отворете Tema04.txt." << endl;
}

// Прочитаме файла обратно, за да се види резултатът на конзолата
fstream iFile("Tema04.txt", ios_base::in);
if (iFile)
{
    cout << "---- Съдържание на Tema04.txt ----" << endl;
    string strLine;
    while (getline(iFile, strLine))
        cout << " " << strLine << endl;
    iFile.close();
}

// ===== 4. Приятелска функция за конзолен вход =====
cout << endl << "===== 4. ConsoleInputEGN =====" << endl;
cout << " преди: " << studYng << endl;
ConsoleInputEGN(studYng);
cout << " след: " << studYng << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

Изходът по-долу е при въведено 9005157895 на въпроса накрая.

```
===== 1. Оператор << за CAddress =====
  addrVarna:   Varna 9010, Studentska #1
  addrSofia:   Sofia 1000, Tsar Osvoboditel #12
  addrDefault: (няма град) 0000, (няма улица)
  верижно: Varna 9010, Studentska #1 | Sofia 1000, Tsar Osvoboditel #12

===== 2-3. Оператор << за CPerson =====
  Иван Петров (мъж), ЕГН: 7503081239, роден: 08.03.1975
  Мария Иванова (жена), ЕГН: 0147222342, роден: 22.07.2001
  Неизвестен (неопределен), ЕГН: (няма), роден: (няма)
--- Разлагане на ЕГН (през GetBirthDate) ---
  7503081239 -> 08.03.1975 (ключ 19750308)
  8512034566 -> 03.12.1985 (ключ 19851203)
  9005157895 -> 15.05.1990 (ключ 19900515)
  0147222342 -> 22.07.2001 (ключ 20010722)

===== 5. Оператор << за CStudent =====
  Георги Стоянов (мъж), ЕГН: 7503081239, роден: 08.03.1975, ф.н.: 21621301, адрес: Varna 9010,
  Studentska #1, оценки: 5.50 4.00 | ср. успех: 4.75
  Елена Димитрова (жена), ЕГН: 8512034566, роден: 03.12.1985, ф.н.: 21621302, адрес: Sofia
  1000, Tsar Osvoboditel #12, оценки: 6.00 5.00 | ср. успех: 5.50
  Петър Колев (мъж), ЕГН: 0147222342, роден: 22.07.2001, ф.н.: 21621303, адрес: Varna 9010,
  Studentska #1, оценки: 3.00 | ср. успех: 3.00

===== 6. Сравнение по възраст =====
  studOld  роден 08.03.1975
  studMid  роден 03.12.1985
  studYng  роден 22.07.2001
  studSame роден 15.05.1990
  studTwin роден 15.05.1990
--- Резултати ---
  studSame == studTwin (една дата): да
  studOld  == studYng:                не
  studOld  != studYng:                да
  studYng  < studOld (по-млад):       да
  studOld  < studYng:                не
  studOld  >= studYng (по-възрастен или равен): да
  studSame >= studTwin (равни):      да
--- Подредба по възраст (най-възрастният първи) ---
  1. Георги Стоянов - 08.03.1975
  2. Елена Димитрова - 03.12.1985
  3. Анна Тодорова - 15.05.1990
  4. Стефан Николов - 15.05.1990
  5. Петър Колев - 22.07.2001

===== 8. Извеждане във файл Тема04.txt =====
  Записани 5 реда. Отворете Тема04.txt.
--- Съдържание на Тема04.txt ---
  Студенти, подредени по възраст
  =====
  1. Георги Стоянов (мъж), ЕГН: 7503081239, роден: 08.03.1975, ф.н.: 21621301, адрес: Varna
  9010, Studentska #1, оценки: 5.50 4.00 | ср. успех: 4.75
  2. Елена Димитрова (жена), ЕГН: 8512034566, роден: 03.12.1985, ф.н.: 21621302, адрес: Sofia
  1000, Tsar Osvoboditel #12, оценки: 6.00 5.00 | ср. успех: 5.50
```

3. Анна Тодорова (жена), ЕГН: 9005157895, роден: 15.05.1990, ф.н.: 21621304, адрес: Sofia 1000, Tsar Osvoboditel #12, оценки: 4.50 | ср. успех: 4.50

4. Стефан Николов (мъж), ЕГН: 9005157909, роден: 15.05.1990, ф.н.: 21621305, адрес: Varna 9010, Studentska #1, оценки: (няма)

5. Петър Колев (мъж), ЕГН: 0147222342, роден: 22.07.2001, ф.н.: 21621303, адрес: Varna 9010, Studentska #1, оценки: 3.00 | ср. успех: 3.00

===== 4. ConsoleInputEGN =====

преди: Петър Колев (мъж), ЕГН: 0147222342, роден: 22.07.2001, ф.н.: 21621303, адрес: Varna 9010, Studentska #1, оценки: 3.00 | ср. успех: 3.00

Въведете ЕГН за Петър Колев: 9005157895

Прието. Нова дата на раждане: 15.05.1990

след: Петър Колев (мъж), ЕГН: 9005157895, роден: 15.05.1990, ф.н.: 21621303, адрес: Varna 9010, Studentska #1, оценки: 3.00 | ср. успех: 3.00

Чести грешки

Код	Съобщение	Причина и решение
C2039	'operator <<': is not a member of 'CAddress'	Написано е <code>CAddress::operator<<</code> . Функцията е глобална — махнете <code>CAddress::</code> .
C2248	cannot access private member	Липсва <code>friend</code> обявяването в класа, или е с различна сигнатура от дефиницията. Двете трябва да съвпадат дума по дума.
C2662	cannot convert 'this' pointer from 'const CStudent' to 'CStudent &'	Извиква се неконстантна функция върху <code>const</code> обект. Проверете дали <code>GetAverage</code> , <code>GetBirthKey</code> и <code>GetBirthDate</code> са маркирани с <code>const</code> .
C2679	binary '<<': no operator found which takes a right-hand operand of type 'CStudent'	Операторът е дефиниран след <code>main</code> , или сигнатурата не съвпада (например липсва <code>const</code>).
C2804	binary 'operator <<' has too many parameters	Операторът е обявен като член на класа с два параметъра. Като член той приема само един — затова трябва да е глобален.
C4996	'sprintf': This function or variable may be unsafe	Използвайте <code>sprintf</code> вместо <code>ostringstream</code> . Или го сменете, или добавете <code>#define _CRT_SECURE_NO_WARNINGS</code> преди всички <code>#include</code> .
—	Stack overflow при извеждане на студент	В <code>operator<<</code> за <code>CStudent</code> липсва <code>static_cast<const CPerson&></code> и операторът вика себе си.
—	Сортирането подрежда наопаки	Посоката на <code>operator<</code> . По-малък ключ = по-възрастен (стъпка 7).
—	Тема04.txt е празен	Липсва <code>oFile.close()</code> , или програмата е прекратена принудително преди края. Данните стоят в буфер, докато потокът не бъде затворен.
—	Програмата „увисва“ накрая	Това е <code>ConsoleInputEGN</code> — чака да въведете ЕГН и да натиснете <code>Enter</code> .

Обобщение: кога член, кога глобална функция

Оператор	Как се пише	Защо
<code>=</code> , <code>[]</code> , <code>()</code> , <code>-</code> <code>></code>	задължително член	Езикът го изисква изрично.
<code><<</code> , <code>>></code>	задължително глобална	Лявата страна е поток, не вашият клас.
<code>==</code> , <code>!=</code> , <code><</code> , <code>>=</code>	и двете са възможни	Глобалната третира двата операнда еднакво. Условието иска глобални.
<code>+</code> , <code>-</code> , <code>*</code>	обикновено глобална	За да работи и <code>2 * obj</code> , не само <code>obj * 2</code> .

Какво следва

Упражнение 5	Огледалните оператори <code>operator>></code> за четене от поток; абстрактен клас <code>CGSM</code> и напълно виртуални функции; множествено наследяване; член-оператор <code>></code> за сравнение по тарифен план; четене на масив от студенти от файл.
---------------------	--